

# Como usar este projeto

---

## D1 — Programação Python para IA · Especialização em Desenvolvimento de Agentes Inteligentes · UTFPR

Este documento é o guia do aluno: o que fazer, na ordem, desde a criação do repositório até a conferência de que a entrega fechou. Foi escrito para ser seguido do começo ao fim, sem pular etapa — e para ser lido sozinho depois. O que o projeto é e por que cada peça existe é apresentado em aula pelo professor.

---

### 1. O que é isto e por que existe

Este pacote traz o **entregável da D1** — o repositório que a disciplina seguinte abre no primeiro dia e que as outras dez constroem em cima pelos 15 meses do curso. Ele chega como o **estado final completo**: código, testes, notebook, dados e scripts, prontos e funcionando.

O trabalho do aluno a partir daqui é outro, e é ele que a entrega final da disciplina avalia: fazer o projeto rodar na própria máquina, colocá-lo em um repositório próprio e **entender cada peça a ponto de explicar** — inclusive a página que descreve o domínio do projeto (seção 5). O pacote garante a fundação; a compreensão é o que o aluno acrescenta — e a demonstra **por escrito**, com a mesma nota de quem constrói cada peça do zero.

Duas promessas deste pacote:

- **Nada do que chega aqui precisa ser escrito do zero.** O código do `src/ams/`, os testes, o notebook, os scripts e o `dominio.md` (seção 5) chegam prontos e funcionam.
  - **Compreensão não chega pronta de propósito.** O pacote entrega a fundação; é o aluno que a entende a ponto de explicá-la — e é isso que a disciplina seguinte consome no primeiro dia.
- 

### 2. O repositório isolado

O repositório que nasce aqui é o do Projeto Integrador pelos 15 meses seguintes. Ele precisa ser **novo e só para isto**:

- **Não** é uma pasta dentro de outro projeto;
- **Não** é uma cópia derivada de outro repositório ( `git clone` de algo que já existia, fork, download de ZIP do GitHub);
- **Não** é uma pasta no Drive — o Drive guarda arquivos, não história de versões, e as disciplinas seguintes fazem `git clone` deste repositório.

Por que isolado: a disciplina seguinte faz o primeiro commit dela em cima deste repositório e assume que ele contém só o projeto; a auditoria do fim do curso percorre o histórico inteiro procurando a evolução real. História misturada com outros projetos contaminada essa leitura — e o histórico é justamente o que o aluno apresenta na defesa.

## 2.1 Criar a pasta e iniciar o repositório

Extraia o `.zip` deste pacote na pasta nova e vazia `C:\CEDAGI\meu-projeto-integrador` (botão direito → Extrair tudo... → digite esse caminho no destino). Use essa pasta, e não Documentos, OneDrive ou Área de Trabalho: o Windows limita o tamanho dos caminhos, o ambiente virtual cria arquivos com até 160 caracteres dentro do projeto, e numa pasta funda ou com acento a instalação falha no meio (o setup confere e para antes). No PowerShell, dentro da pasta nova:

```
git init
git add .
git commit -m "Início do Projeto Integrador"
git branch -M main
```

O que cada comando fez: `git init` transformou a pasta em repositório; `git add .` marcou todos os arquivos do pacote para o primeiro registro; `git commit` gravou esse registro na história; `git branch -M main` deu ao ramo principal o nome `main`, que é o que o GitHub espera. No Windows é normal o `git add` imprimir vários avisos sobre "LF will be replaced by CRLF": são de conversão de quebra de linha, não são erro.

Se o `git commit` reclamar que falta identidade, configure uma vez só (valerá para todos os projetos da máquina):

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

## 2.2 Levar o repositório para o GitHub

O repositório precisa estar acessível para a correção e para as disciplinas seguintes. No GitHub ([github.com](https://github.com)), com a conta do aluno, crie um repositório **vazio** (sem README, sem licença, sem `.gitignore` — tudo isso já chega no pacote) e volte ao terminal:

```
git remote add origin https://github.com/SEU-USUARIO/NOME-DO-REPOSITORIO.git
git push -u origin main
```

O `git push` publica a história local no GitHub; o `-u` conecta o ramo local ao remoto, e os `push` seguintes são só `git push`.

**A regra do link:** o link do repositório precisa abrir, em uma **aba anônima**, para quem não é o dono.

Repositório público já satisfaz — e é o recomendado pelo curso, porque o portfólio é parte da validação do fim do curso. Repositório privado também serve, desde que compartilhado com a conta de correção que o professor anunciar no ambiente da disciplina.

## 2.3 A conferência de que o `.env` ficou de fora

O pacote traz um `.gitignore` que já cobre `.env`, `.venv/` e `__pycache__/`. Para confirmar que o arquivo de configuração com a chave não entrou na história:

```
git status
```

Se nada aparecer listado (além, talvez, do aviso de ramo à frente), o `.env` está protegido. Volte a esta checagem toda vez que o `git status` mostrar algo inesperado antes de um `git add`.

### 3. A chave do Google

O projeto chama um modelo de linguagem (Gemini Flash) pela API, e a chave é **sempre do aluno** — gratuita, sem cartão de crédito. Os passos abaixo reproduzem o `GUIA-DE-INSTALACAO.md` do curso:

1. Acesse <<https://aistudio.google.com>> com qualquer conta Google.
2. Menu lateral → **Get API key** → **Create API key** (crie um projeto se for pedido).
3. Copie a chave exibida **inteira**, do primeiro ao último caractere. O Google emite chave em mais de um formato (umas começam com `AIza`, outras com `AQ.`) — qualquer um deles serve.

Na raiz do repositório, copie o modelo de configuração para o arquivo real:

```
copy .env.example .env
```

(No macOS/Linux: `cp .env.example .env`.) Abra o `.env` em qualquer editor e cole a chave na linha indicada:

```
GEMINI_API_KEY=cole-aqui-a-sua-chave-inteira-sem-aspas
```

Salve. Duas regras que valem pelo resto do curso:

- **.env nunca vai para o repositório** — o `.gitignore` já cuida disso (seção 2.3). O `.env.example` segue no repositório com `GEMINI_API_KEY=` vazio: ele documenta quais variáveis o projeto espera, sem segredo algum.
- **A chave é pessoal.** Ela é a senha da cota gratuita da conta de quem a criou; não compartilhe, não cole em outro arquivo, não envie a ninguém.

O modelo padrão já vem configurado no `.env.example` (`GEMINI_MODEL=gemini-3.5-flash-lite`) e não precisa ser mudado.

### 4. O ambiente, script a script

Todos os comandos desta seção rodam **na raiz do repositório** — a pasta que contém `src/`, `tests/` e `scripts/`. A ordem importa: cada passo depende do anterior. Os exemplos são do PowerShell do Windows; os equivalentes para macOS/Linux aparecem em cada passo.

#### 4.1 `setup-windows.ps1` / `setup-unix.sh` — o instalador

```
powershell -ExecutionPolicy Bypass -File scripts\setup-windows.ps1
```

(macOS/Linux: `bash scripts/setup-unix.sh`)

O que faz: procura um Python 3.12+, cria o ambiente virtual `.venv`, instala os pacotes do `requirements.txt` com versões fixadas, instala o pacote `ams` do projeto, cria o `.env` a partir do `.env.example` se ele não existir e roda a verificação final de ambiente. Pode ser rodado de novo sem problema — e **preserva o** `.env` que já tem a chave.

Quando dá certo, o fim da saída mostra cada item da verificação com  e a frase "**Todos os itens críticos passaram. Seu ambiente está pronto!**". Quando algo falha, o item aparece com  seguido da instrução exata do que fazer — siga-a e rode o script de novo. Não exige privilégio de administrador; em máquina corporativa, se a instalação dos pacotes falhar, o firewall está bloqueando o acesso ao repositório de pacotes — peça liberação à TI e avise o professor.

## 4.2 `verifica-ambiente.py` — o diagnóstico

O instalador já roda este script no fim. Para rodá-lo de novo, a qualquer momento:

```
.venv\Scripts\python.exe scripts\verifica-ambiente.py
```

(macOS/Linux: `.venv/bin/python scripts/verifica-ambiente.py`)

O que faz: confere versão do Python, pacotes instalados, variáveis do `.env`, conectividade com a API do Gemini e permissões de escrita. Cada  vem com "o que fazer". É o script que separa problema de máquina de problema de projeto: se ele está todo verde e algo falha adiante, o problema é outro. Um detalhe que poupa susto: o item da chamada de teste ao modelo pode falhar com HTTP 429 (cota estourada) ou HTTP 503 (modelo sobrecarregado no momento) — os dois são transitórios; espere alguns minutos e rode de novo.

## 4.3 `baixar-dados.py` — os dados completos

```
.venv\Scripts\python.exe scripts\baixar-dados.py
```

(macOS/Linux: `.venv/bin/python scripts/baixar-dados.py`)

**Este passo precisa de rede**, e é o que fecha a entrega: o pacote traz uma amostra de 200 linhas para o projeto funcionar desde o primeiro minuto, mas o Contrato de Saída exige  $\geq 500$  registros. O script baixa o dataset completo (10.000 registros), confere a integridade e grava a proveniência em `data/raw/PROVENIENCIA.txt`. Sem este passo com rede, o repositório roda, os testes passam — e a entrega, ainda assim, não está fechada: o `verifica-entrega.py` aceita a dupla "amostra + script de download" como promessa, mas a análise que a rubrica avalia e que as disciplinas seguintes treinam é a do dataset completo. Dito sem rodeio: **rode este script com internet antes de entregar**.

Quando dá certo, o fim da saída diz OK: `data\raw\ai4i2020.csv` (10000 linhas, 14 colunas) e a fonte usada. Se a rede falhar, o script tenta fontes alternativas e, na pior hipótese, avisa com instrução — rode de novo quando a conexão voltar. Rode duas vezes e ele não refaz nada (é idempotente); `--forçar` refaz de propósito.

## 4.4 pytest — os testes

Com o ambiente ativado:

```
.venv\Scripts\activate.bat  
pytest
```

(PowerShell: `.venv\Scripts\Activate.ps1` e depois `pytest`; macOS/Linux: `source .venv/bin/activate` e depois `pytest`)

O que faz: roda os testes de `tests/` — todos **sem rede**. Quando dá certo, a última linha diz `70 passed` — depois do passo 4.3, com o dataset completo já baixado; antes dele, `69 passed` e um teste marcado como `skipped`, que é justamente o que só roda com o dataset completo (o número pode crescer quando o aluno acrescenta testes próprios; o importante é `passed` e nenhum `failed`). Quando falha, o teste quebrado aparece com o motivo — e a primeira suspeita deve ser mudança recente no código, não o pacote.

## 4.5 python -m ams.cliente\_llm — o cliente do modelo

```
python -m ams.cliente_llm
```

O que faz: pega os primeiros 20 registros do dataset, transforma cada um em uma ordem de serviço, envia ao modelo, valida cada resposta como objeto `AvaliacaoRisco` (Pydantic), tenta de novo quando a rede ou a cota falham e registra cada avaliação em `data/logs/ams.jsonl`. No fim, imprime uma tabela com risco, bloqueio recomendado, normas e justificativa por ordem, e a linha de resumo do tipo `✅ 18/20 avaliadas · 🚫 18 bloqueio(s) recomendado(s) · 0.2 s · 93.8 ordens/s`. Com as fixtures de fábrica, **duas ordens falham de propósito** (as linhas com `❌ falhou`): são os casos didáticos de erro de esquema e de limite de cota, que existem para exercitar o tratamento de erro sem gastar cota — o comando termina bem mesmo assim, e o detalhe de cada falha está no log. No modo online, ordens cujas tentativas se esgotam — por exemplo num pico de demanda do modelo, que responde 503 — aparecem do mesmo jeito; rodar de novo mais tarde costuma resolver.

Este passo **gasta cota gratuita** (20 chamadas). Quando a cota apertar — ou para trabalhar sem internet — use o modo sem rede, que lê respostas gravadas em `data/fixtures/` em vez de chamar a API:

```
set AMS_MODO_OFFLINE=1  
python -m ams.cliente_llm  
set AMS_MODO_OFFLINE=
```

(PowerShell: `$env:AMS_MODO_OFFLINE = "1"` antes e `Remove-Item Env:AMS_MODO_OFFLINE` depois; macOS/Linux: `AMS_MODO_OFFLINE=1 python -m ams.cliente_llm`)

No modo sem rede a saída diz `modo: 📄 OFFLINE (fixtures)` e o resto é igual — mesma validação, mesmo log, mesma tabela. **Quando salva a aula**: cota estourada, internet instável, máquina corporativa sem saída para a API. **Quando não serve**: para conferir que a chave do aluno funciona e para qualquer pergunta cuja resposta dependa do modelo de verdade — o modo sem rede repete as respostas gravadas, que foram feitas para as ordens de serviço da manutenção. A variável também pode ser fixada no `.env`

(descomentando a linha `# AMS_MODALITY=1`); o `set AMS_MODALITY=` acima vale só para a janela atual.

#### 4.6 `verifica-entrega.py` — a conferência final

```
python scripts\verifica-entrega.py
```

O que faz: percorre o Contrato de Saída item a item — estrutura de pastas, Git, `.env` fora do versionamento, requirements com versões, README, `dominio.md`, dataset, notebook, funções tipadas, testes e o cliente rodando sem rede. Cada **✗** crítico vem com "o que fazer". A entrega fecha quando **nenhum ✗ crítico** resta (seção 6). Não avalia qualidade nem dá nota — isso é a rubrica, com o professor.

### 5. O que você precisa dominar: o `dominio.md`

O `dominio.md` na raiz do repositório descreve o domínio do projeto em uma página, respondendo seis perguntas: o problema, a decisão a automatizar, o usuário, a fonte de dados, por que um agente é solução e como saber se está funcionando. Ele chega **preenchido** com o domínio padrão do pacote — manutenção industrial, com o dataset *AI4I 2020* (o mesmo dos dados, do notebook e dos testes).

Este é o arquivo mais importante do pacote, e o trabalho do aluno com ele é **ler e entender cada resposta**. A disciplina seguinte abre este arquivo no primeiro dia e o transforma na especificação do agente que será construído, medido e — no fim do curso — auditado; o professor vai pressupor que você conhece o problema, a decisão e os dados daqui. Se algum trecho não ficar claro, volte ao código e ao notebook que o sustentam — a seção 4 mostrou como rodar cada peça.

**Quem escolheu outro dataset troca também o domínio.** As seis respostas deste arquivo descrevem a manutenção industrial; se o seu domínio é outro, reescreva as respostas com as suas palavras, mantendo as seis perguntas — o `COMO-TROCAR-O-DATASET.pdf` orienta a troca da fonte de dados, e os requisitos mínimos conferidos pelo `verifica-entrega.py` (arquivo presente na raiz, com problema, decisão a automatizar e fonte de dados) valem para qualquer domínio.

Se você alterou o arquivo, grave a versão sua no repositório:

```
git add dominio.md
git commit -m "Domínio do Projeto Integrador"
git push
```

### 6. Como conferir que fechou

Dois testes, nesta ordem:

**1. O `verifica-entrega.py` sem nenhum ✗ crítico** (seção 4.6). Cada **✗** crítico que restar vem com a instrução do que fazer — resolva e rode de novo. Itens marcados como "recomendado" não impedem a entrega, mas enfraquecem a rubrica: vale a pena resolvê-los também.

**2. O teste do clone novo** — feito na véspera de entregar. Numa pasta **fora** do repositório, clone o que foi publicado no GitHub e siga o README à risca:

```
cd C:\CEDAGI
git clone https://github.com/SEU-USUARIO/NOME-DO-REPOSITORIO.git teste-clone
cd teste-clone
copy C:\CEDAGI\meu-projeto-integrador\.env .env
powershell -ExecutionPolicy Bypass -File scripts\setup-windows.ps1
.venv\Scripts\python.exe scripts\baixar-dados.py
.venv\Scripts\activate.bat
pytest
python -m ams.cliente_llm
```

O `copy` do `.env` traz a chave de quem testa para o clone sem digitá-la de novo — sem ele, o cliente do modelo roda e encerra com todas as ordens falhando, porque a chave do clone é vazia (o `setup` a cria a partir do `.env.example`, que vem com `GEMINI_API_KEY=` vazio de propósito).

Se o clone novo instala, baixa os dados, passa nos testes e roda o cliente **sem editar nenhum arquivo**, o repositório está entregável. É exatamente o que a correção fará com ele — e o que a disciplina seguinte fará no primeiro laboratório.

---

*Um último parágrafo, para quem leu até aqui sozinho: este repositório não é uma tarefa que termina em duas semanas. Ele é o sistema que o aluno apresenta na defesa do curso. Cada disciplina que vem acrescenta uma peça em cima dele, e nenhuma recomeça do zero — por isso o cuidado com o clone limpo, com a compreensão do `dominio.md` e com o histórico de commits honesto vale mais do que qualquer apressado de última hora.*